

FITBUDDY – AI FITNESS PLAN GENERATOR USING GEMINI MODELS

4. Project Planning Phase

The Project Planning Phase determines how the FitBuddy project will be developed, tested, and completed. The project was divided into smaller tasks so that each component could be implemented and verified systematically.

The first task was to prepare the development environment. Python and Visual Studio Code were selected as the primary development tools. A Python virtual environment was created to isolate the project dependencies.

The second task was to create the basic project structure. Separate folders were planned for application code, AI modules, templates, static files, and tests. This organization helps maintain the project clearly.

The third task was to install the required Python libraries. Important dependencies include FastAPI for backend development, Uvicorn for running the server, Jinja2 for HTML templates, SQLAlchemy for database operations, Pydantic for data validation, and Google's Gemini SDK for AI integration.

The fourth task was database planning. The User database model was designed to store the user's User ID, name, age, weight, fitness goal, workout intensity, original plan, updated plan, nutrition tip, and timestamps.

The fifth task was AI integration planning. The Gemini workout-generation component was planned separately from the nutrition and feedback components. This allows each AI function to be tested independently.

The sixth task was frontend planning. The main input page would collect the required user information. The result page would display the generated workout and nutrition information and provide a feedback mechanism.

The project development was therefore planned in the following order:

- Environment setup.
- Project folder creation.
- Dependency installation.
- Database configuration.
- Data models and schemas.
- FastAPI backend.
- Gemini AI integration.
- Frontend templates.
- Feedback and updated-plan functionality.

- Testing and debugging.
- Documentation.
- Final demonstration.

This planning approach helps reduce development errors and ensures that each major functionality is completed before moving to the next stage.